



Artificial Intelligence Fundamentals

SEARCH

Problem-solving agents

An agent may need to **plan ahead** to solve a problem

Search algorithm: given a problem, return a **sequence of actions** that form a **path** to a **goal state** (or failure)

This lecture: episodic, single agent, **fully observable**, deterministic, static, discrete environments.

Informed search algorithms vs. **uninformed** search algorithms

- Can the agent estimate how far it is from the goal?

Since we have access to the complete environment, we can search for the solution and then execute it.

A general problem-solving agent

Open-loop (offline): the agent searches for the optimal sequence of action, then it executes it.
(fully-observable, deterministic)

Closed-loop (online): the agent continuously evaluates the feedback from the environment.
(partially-observable, stochastic)

```
function Simple-Problem-Solving-Agent(percept) returns an action
  static: seq, an action sequence, initially empty
           state, some description of the current world state
           goal, a goal, initially null
           problem, a problem formulation

  state ← Update-State(state, percept)
  if seq is empty then
    goal ← Formulate-Goal(state)
    problem ← Formulate-Problem(state, goal)
    seq ← Search(problem)
  action ← Recommendation(seq, state)
  seq ← Remainder(seq, state)
  return action
```

Defining the search problem

State space (it's a **graph**!): all cities

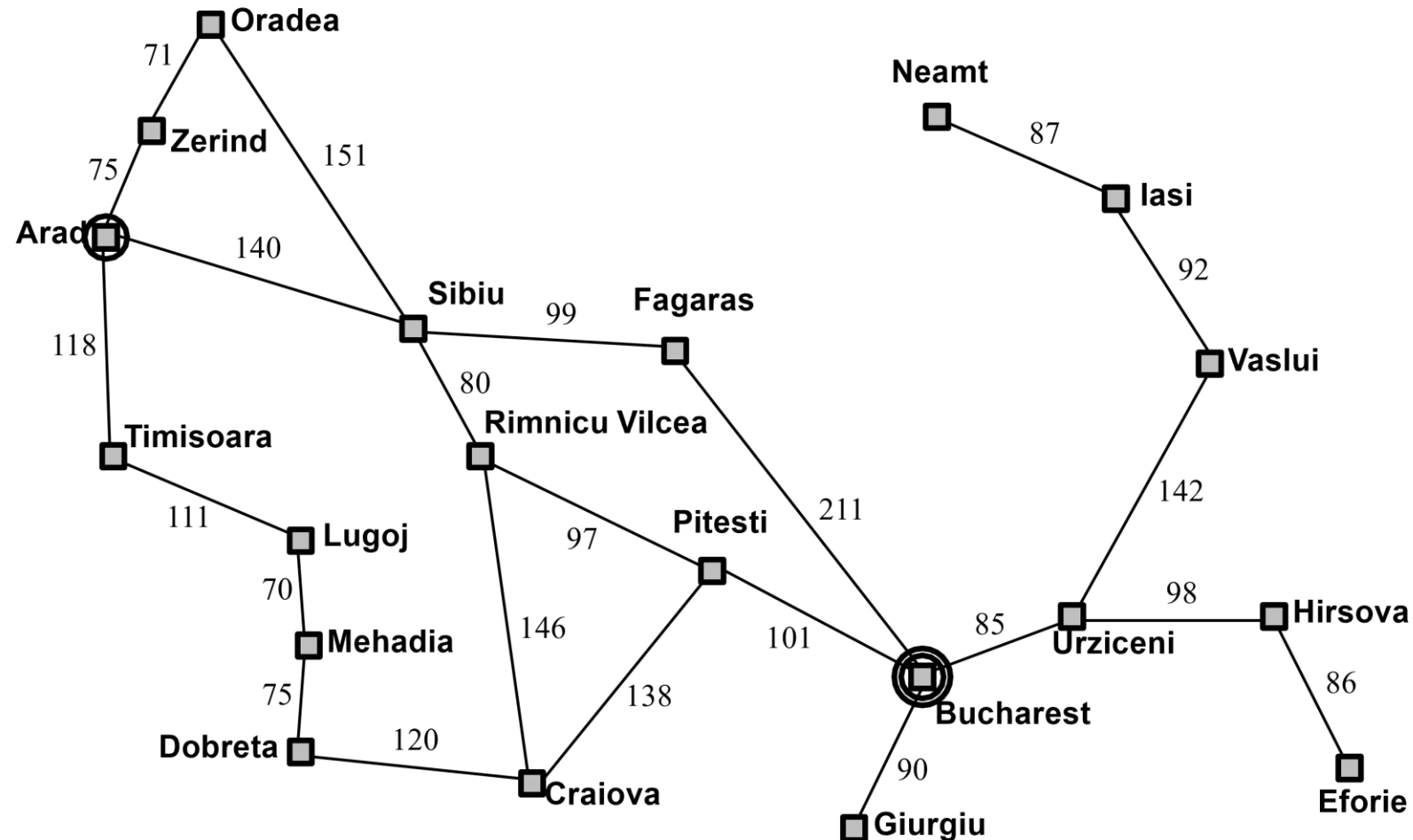
Goal state (≥ 1): Bucharest
(*can also be a function*)

Initial state: Arad

Action:
move between adjacent cities

Path: sequence of actions
(pairs of cities, or simply a sequence of cities)

Solution:
a path from initial to goal state



Defining the search problem

Transition model:

$Result(state, action) = next\ state$

Applicable actions:

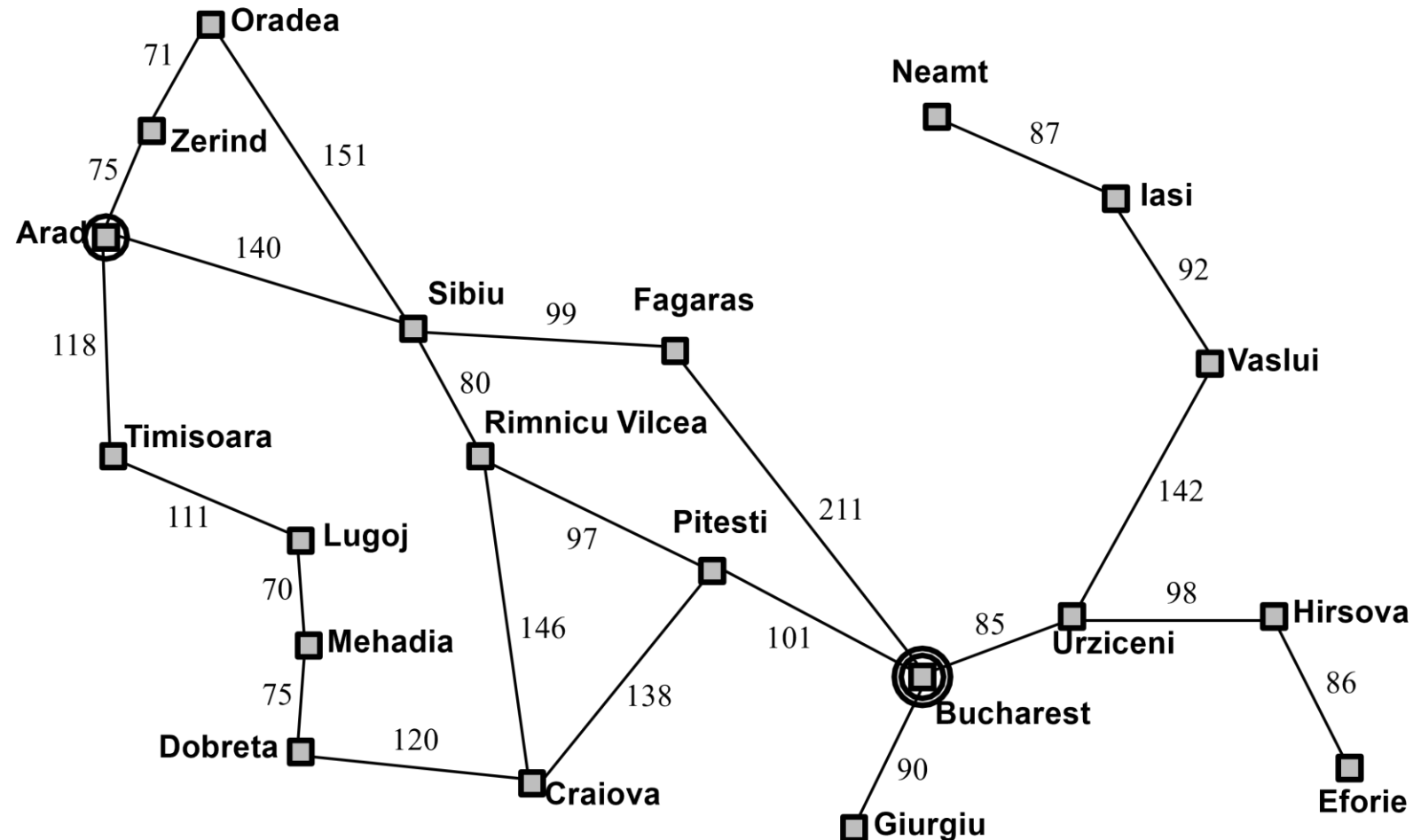
$Actions(Arad) = \{ToZerind, ToTimisoara, ToSibiu\}$

Action-cost function:

$c(state, action, nextstate)$

Should be aligned with performance metric

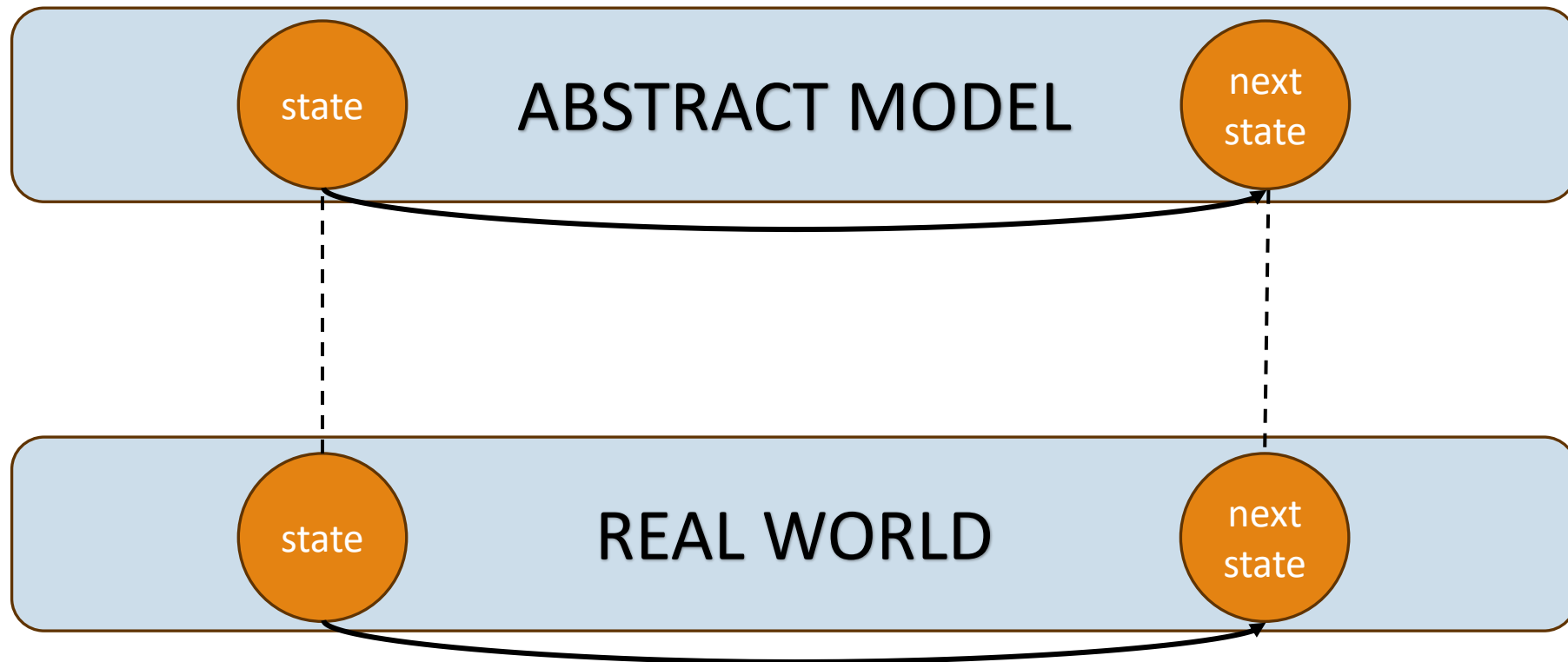
Optimal solution: solution with minimal cost / maximum reward



«A MODEL SHOULD BE AS SIMPLE
AS POSSIBLE, BUT NO SIMPLER»

EINSTEIN (KIND OF)

Modeling the problem



Some design choices: a quick guide

Deterministic, fully observable → single-state problem

- Agent knows exactly which state it will be in: solution is a sequence of states or actions

Non-observable → conformant/sensorless problem

- Agent does not perceive the state; solution (if any) is a sequence of actions

Nondeterministic and/or partially observable → contingency problem

- percepts provide new information about current state
- solution is a contingent plan or a policy
- often interleave search, execution

Back to the basics: the Vacuum world

Single-state, start in #5. Solution?

[Right, Suck]

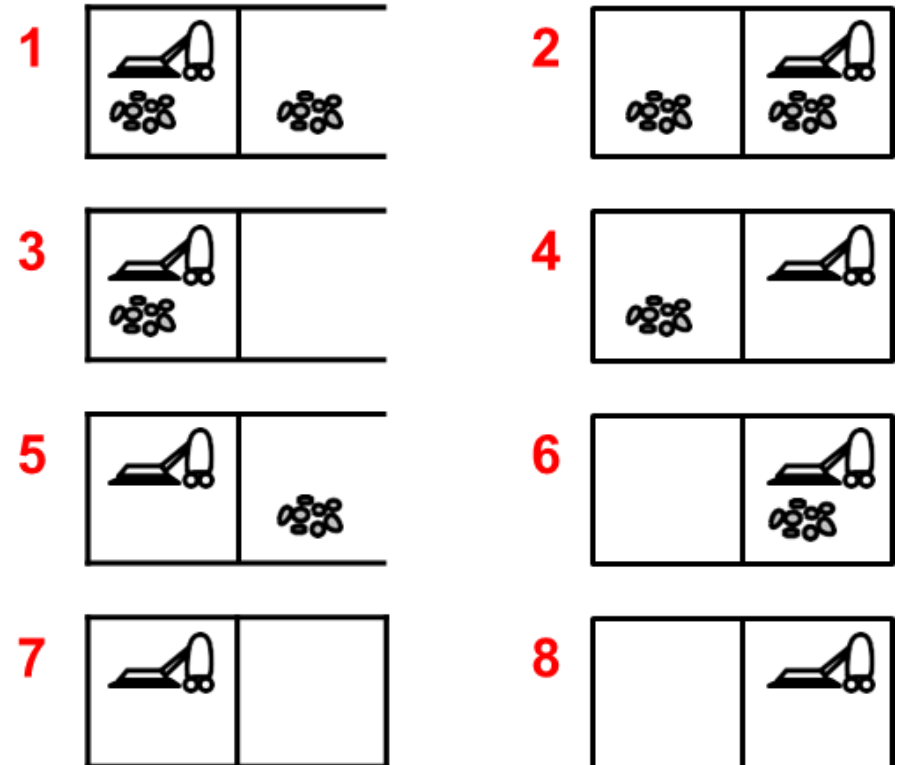
Sensorless, start in {1, 2, 3, 4, 5, 6, 7, 8}. Solution?

[Right, Suck, Left, Suck]

Contingency, start in #5

Partially observable: need to detect dirt. Solution?

[Right, if dirt: Suck]



Vacuum world's state space

states

- integer dirt and robot locations
(ignore dirt amounts etc.)

actions

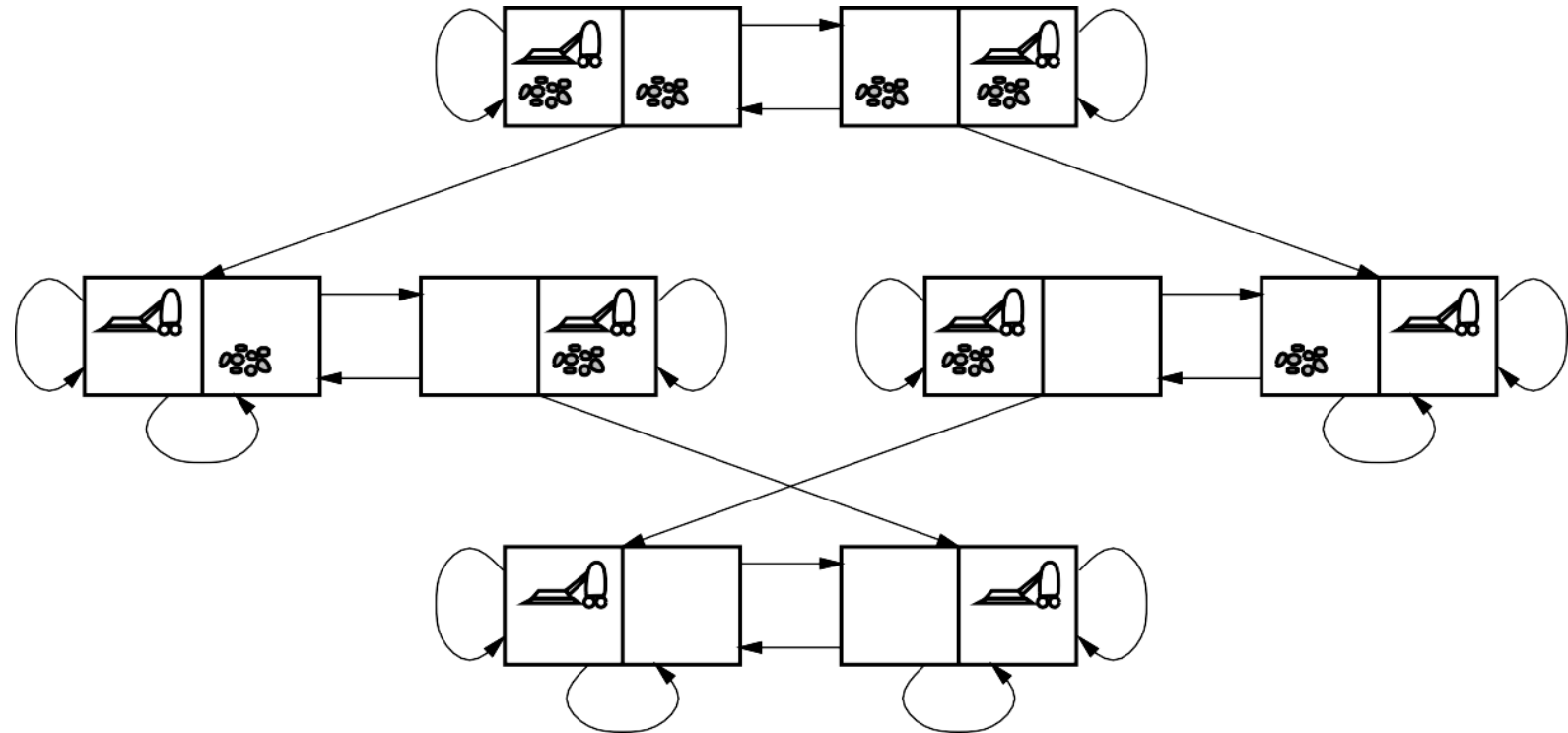
- Left, Right, Suck, NoOp

goal test

- clean(s)*
- (*clean* tests both rooms)

path cost

- 1 per action, 0 for NoOp



The 8-puzzle (n-puzzle is NP-hard)

States

- integer locations of tiles (ignore intermediate positions)

Actions

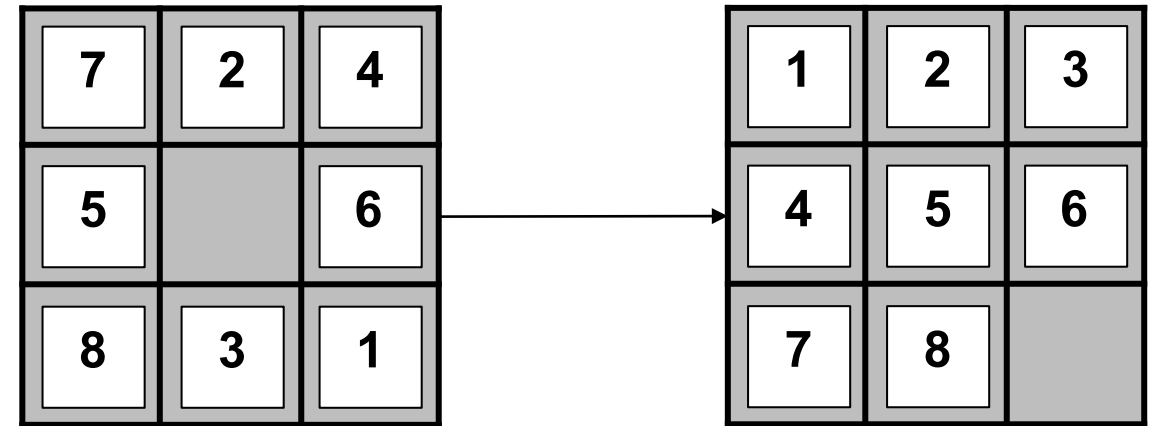
- move blank left / right / up / down

goal test

- goal state (given)

path cost

- 1 per move



Robotic assembly

States

- real-valued coordinates of robot joint angles parts of the object to be assembled

Actions

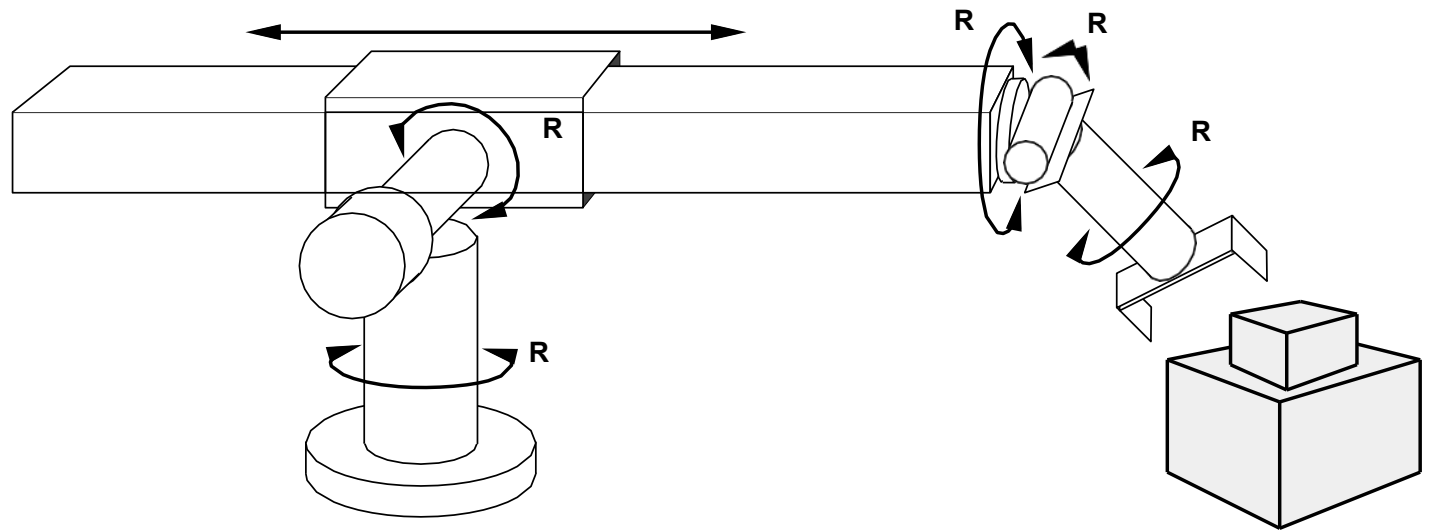
- continuous motions of robot joints

goal test

- complete assembly
- no robot included!

path cost

- time to execute



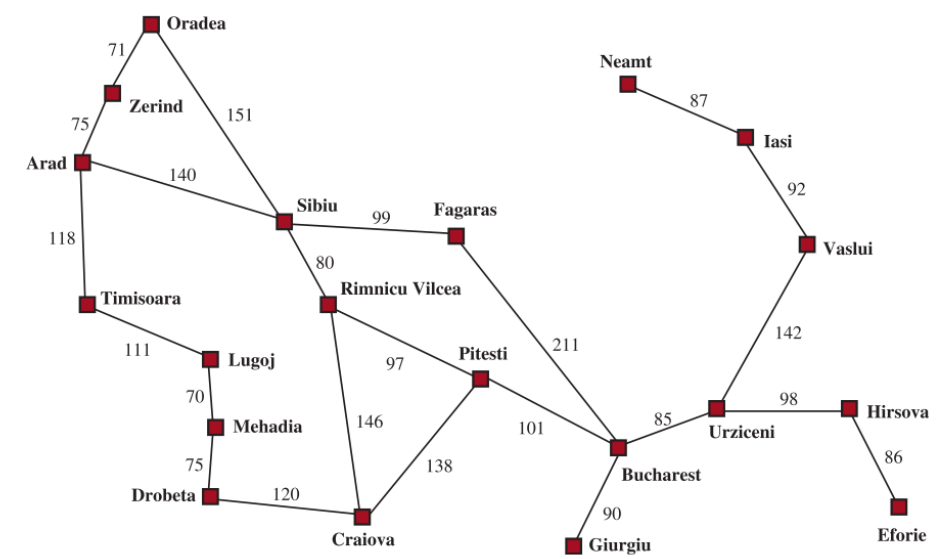
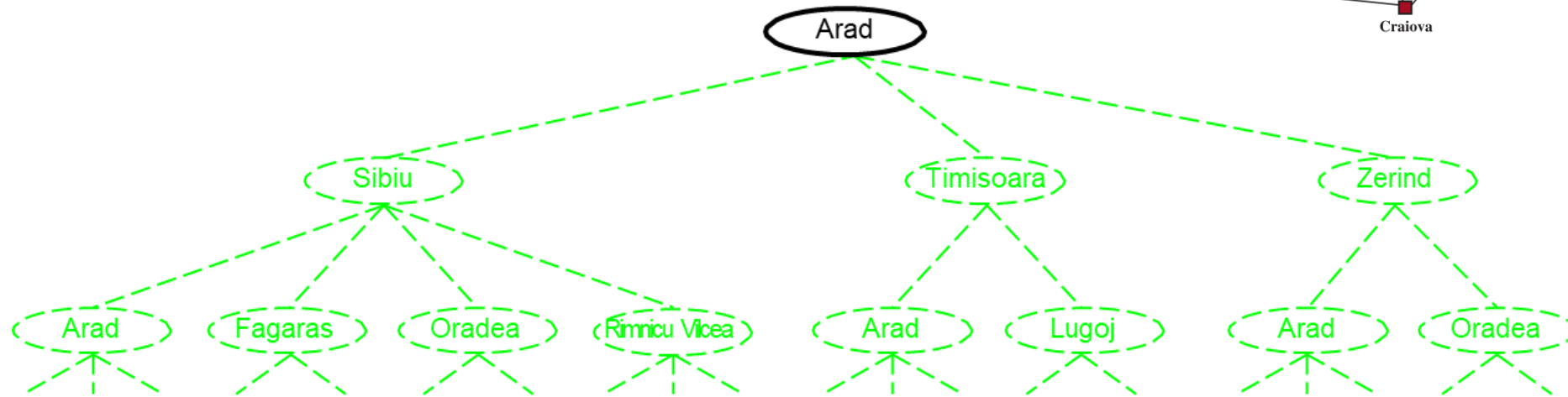
Can we search please?

Tree search: start from already visited states and **expand** them (generate **successor states**)

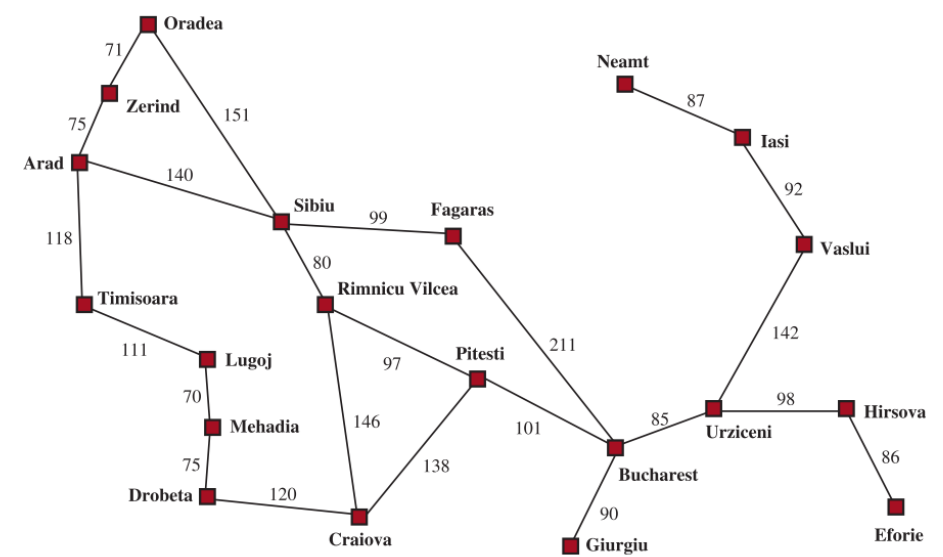
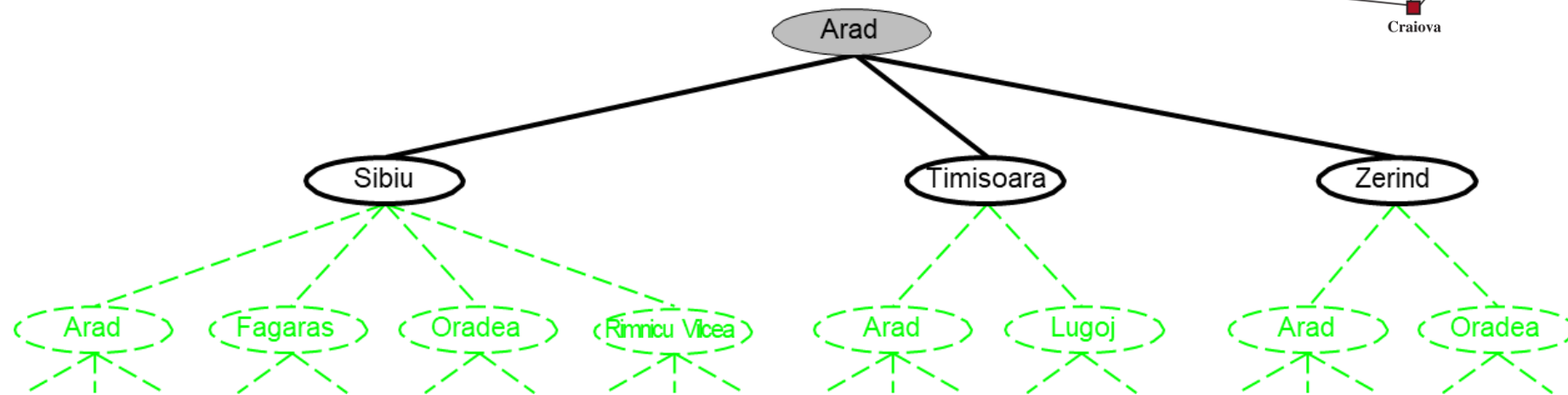
- **Frontier:** set of visited, but unexpanded nodes. It separates expanded nodes from unreached nodes
- **Reached nodes** = frontier + expanded nodes

```
function Tree-Search(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```

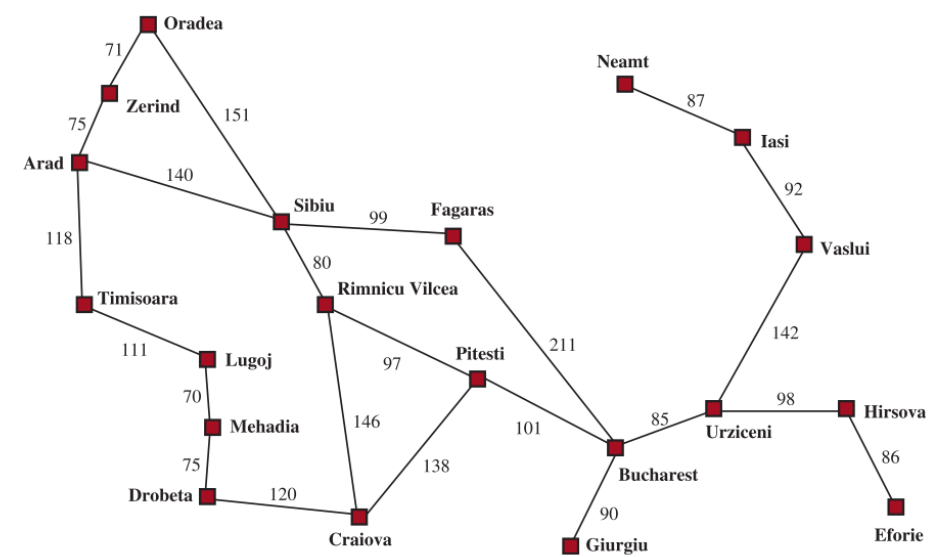
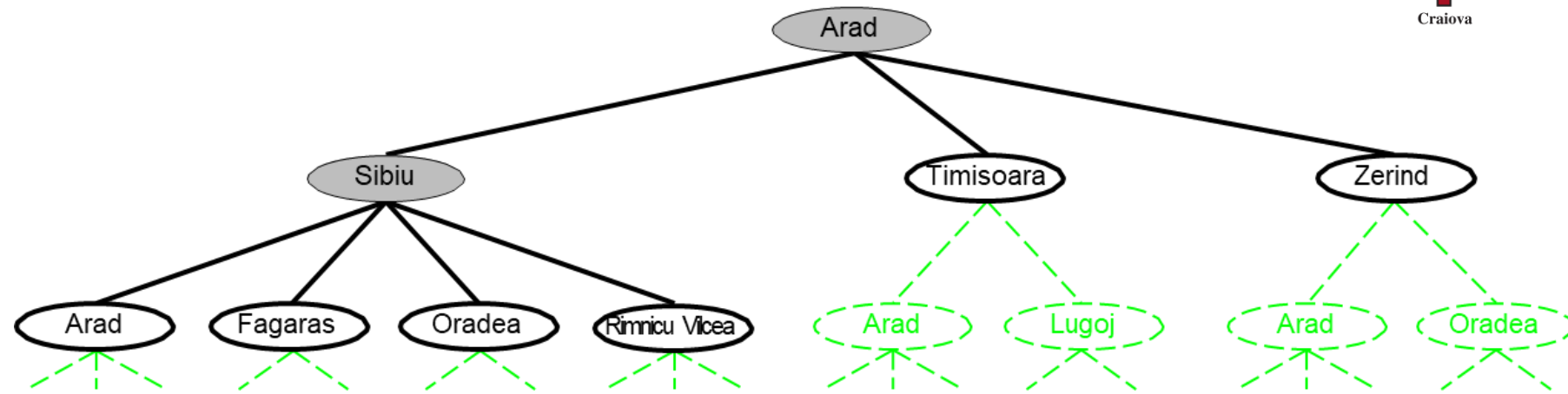
Tree search



Tree search



Tree search



When coding...

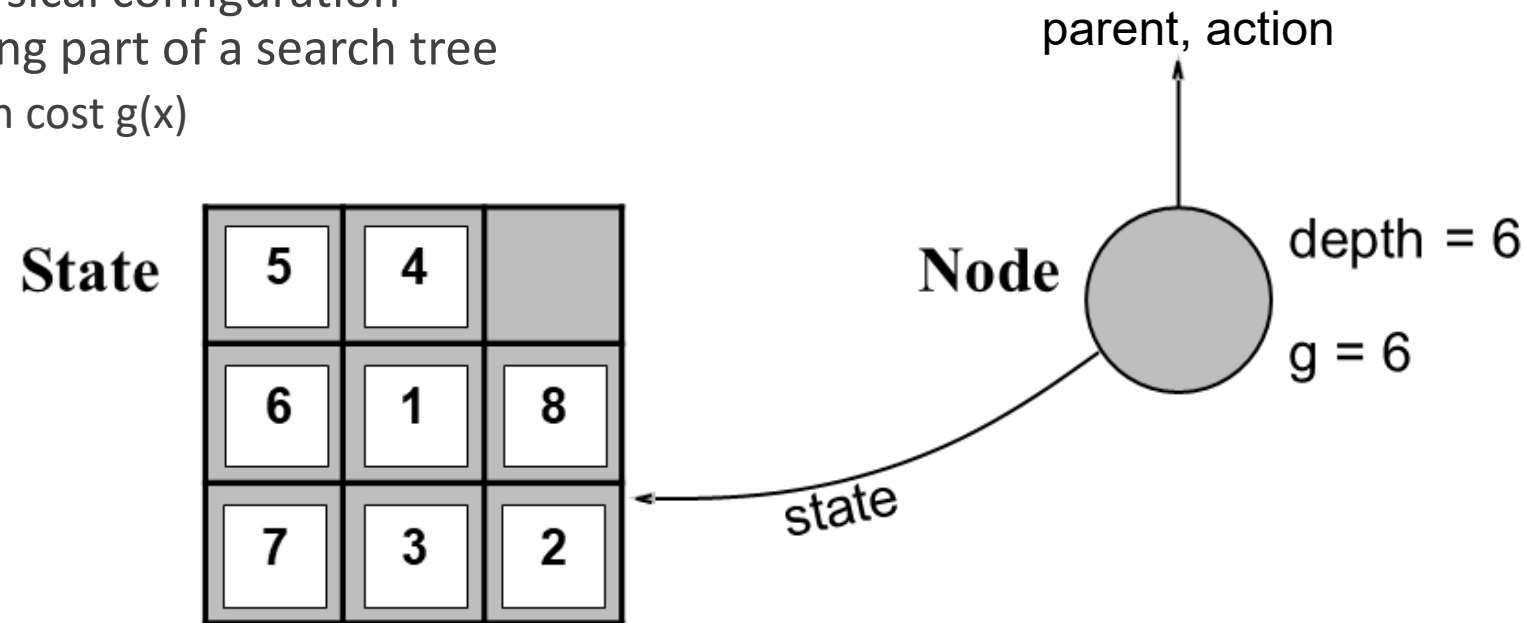
A **state** is a (representation of) a physical configuration

A **node** is a *data structure* constituting part of a search tree

- includes parent, children, depth, path cost $g(x)$

The *Expand* function (see next)

- creates new nodes
- fills in the various fields
- uses the *SuccessorFn* of the problem to create the corresponding states.



State space $\neq$ search tree

Finite state space can lead to infinite search trees (loops, redundant paths)

- See later

Node in a search tree = state in the state space

States are all different in a state space

Nodes are **not** unique in a search tree

- They can represent the same state

Your code needs to take this into account

Choosing a search strategy

A search strategy is defined by **picking the order of node expansion**

Strategies are **evaluated** along the following dimensions:

- **completeness** — does it always find a solution if one exists? Does it always fail if it does not?
- **time complexity** — number of nodes generated/expanded
- **space complexity** — maximum number of nodes in memory
- **optimality** — does it always find a least-cost solution?

Time and space complexity are measured in terms of:

- **b** — maximum branching factor of the search tree
- **d** — depth of the least-cost solution
- **m** — maximum depth of the search tree (may be ∞)

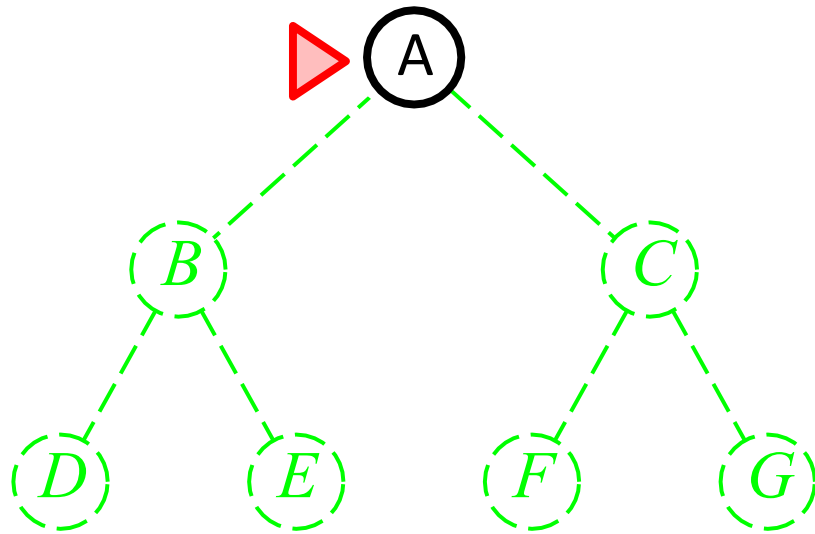
Uninformed tree search

Uninformed strategies use only the information available in the problem definition. You don't know how far away you are from the goal(s).

- Breadth-first search
- Uniform-cost search
- Depth-first search
- Depth-limited search
- Iterative deepening search

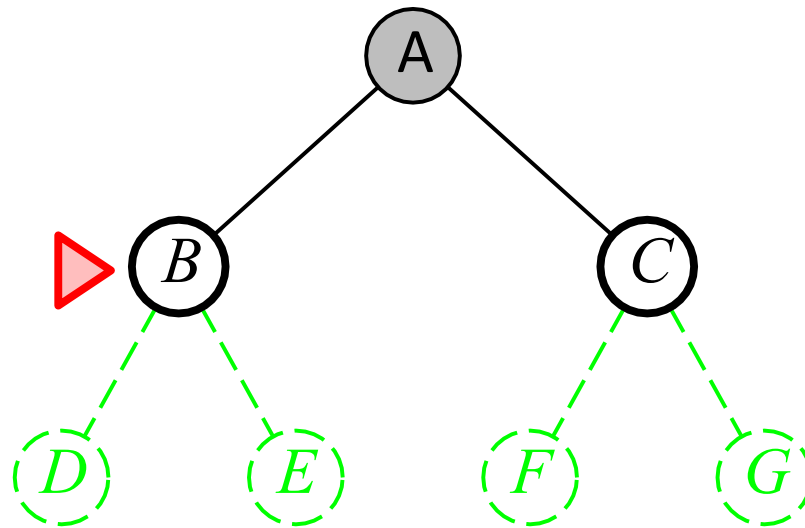
Breadth-first search

Actions have the same cost
Frontier is a FIFO queue



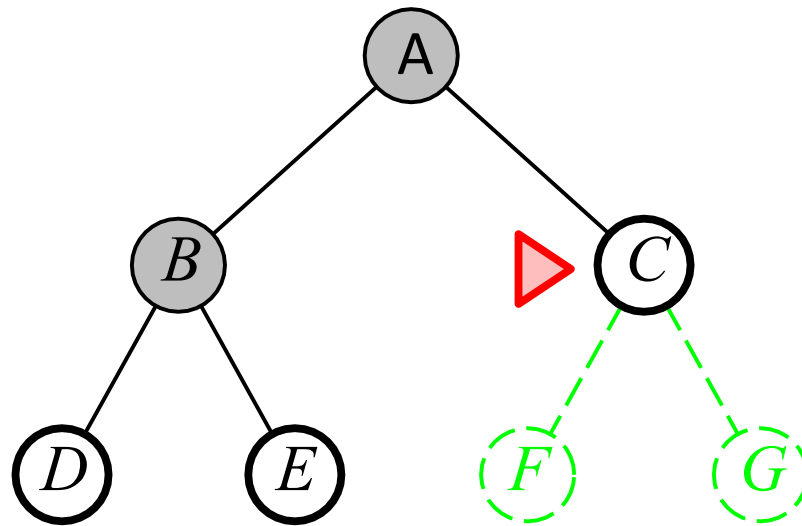
Breadth-first search

Actions have the same cost
Frontier is a FIFO queue

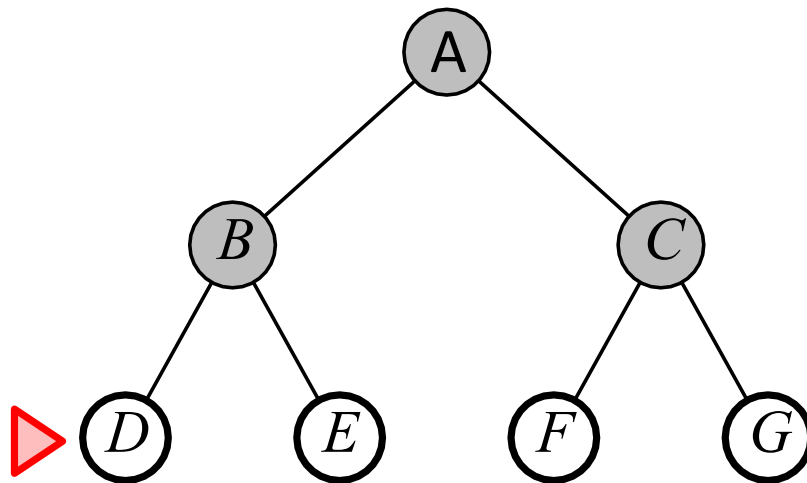


Breadth-first search

Actions have the same cost
Frontier is a FIFO queue



Breadth-first search



Complete? Yes (if b is finite)

Time? $1 + b + b^2 + b^3 + \dots + b^d = O(b^d)$

Space? $O(b^d)$ (keeps every node in memory)

Optimal? Yes (if cost = 1 per step); not optimal in general

Space is the big problem

BFS pseudocode

```
function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure  
  node  $\leftarrow$  NODE(problem.INITIAL)  
  if problem.IS-GOAL(node.STATE) then return node  
  frontier  $\leftarrow$  a FIFO queue, with node as an element  
  reached  $\leftarrow$  {problem.INITIAL}  
  while not IS-EMPTY(frontier) do  
    node  $\leftarrow$  POP(frontier)  
    for each child in EXPAND(problem, node) do  
      s  $\leftarrow$  child.STATE  
      if problem.IS-GOAL(s) then return child  
      if s is not in reached then  
        add s to reached  
        add child to frontier  
  return failure
```

Dijkstra's algorithm: uniform-cost search

BFS but actions have different cost → expand the node with minimal path cost (from root to that node).

Frontier is a weighted queue (lowest path cost first)

Complete? Yes, if minimum action cost is positive ($\epsilon > 0$)

Time? $O(b^{1+\lceil \frac{C^*}{\epsilon} \rceil})$, where C^* is the cost of the optimal path

→ can waste time exploring low-cost paths that lead nowhere (can be $\gg b^d$)

Space? same as time

Optimal? Yes. Nodes expanded in increasing order of path cost

Why +1?

Sibiu (initial state)

→exp. Rimnicu Vilcea

- generates Pitesti

→exp. Fagaras

- generates Bucharest (**does not test for goal**)

→exp. Pitesti

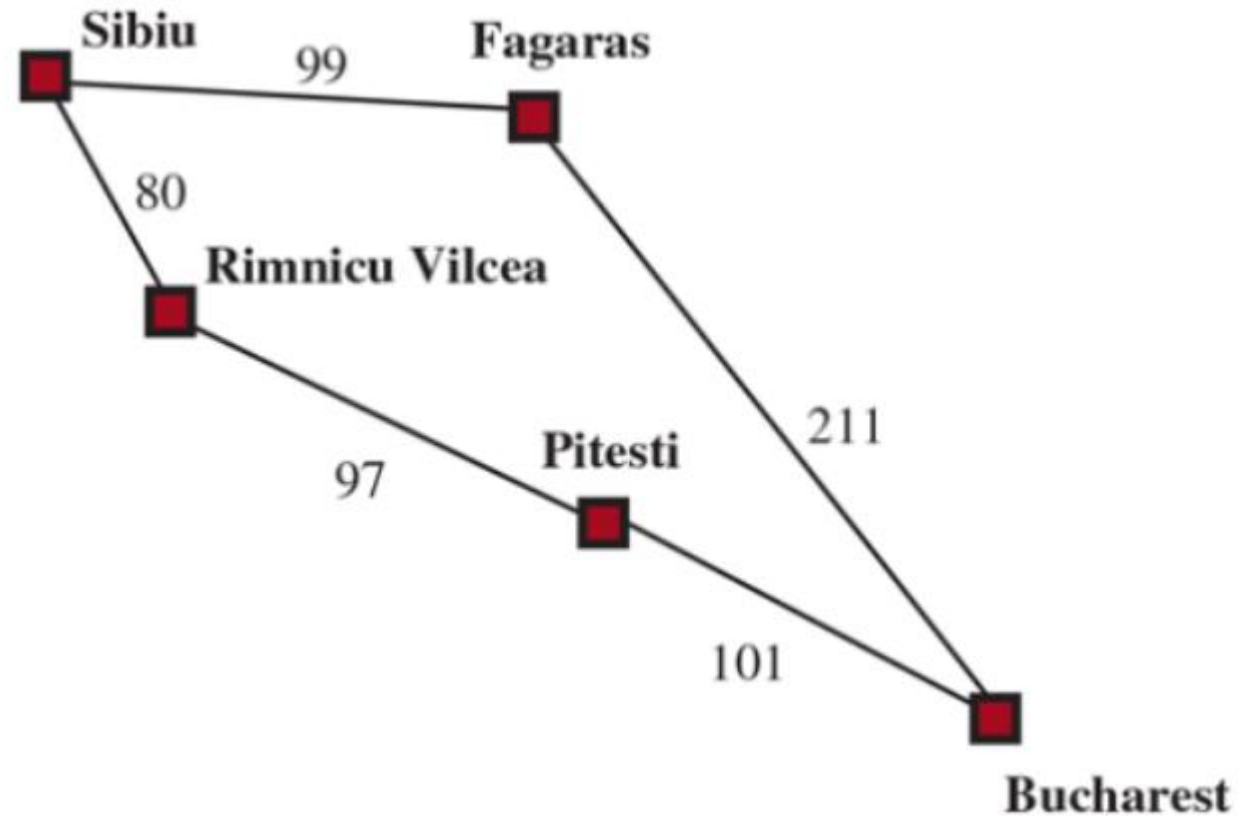
- Generates Bucharest again (**replacement**)

→exp. Bucharest

- **goal found**

Intuition: 1 level more than BFS
which immediately checks for goal

- does not expand the goal node

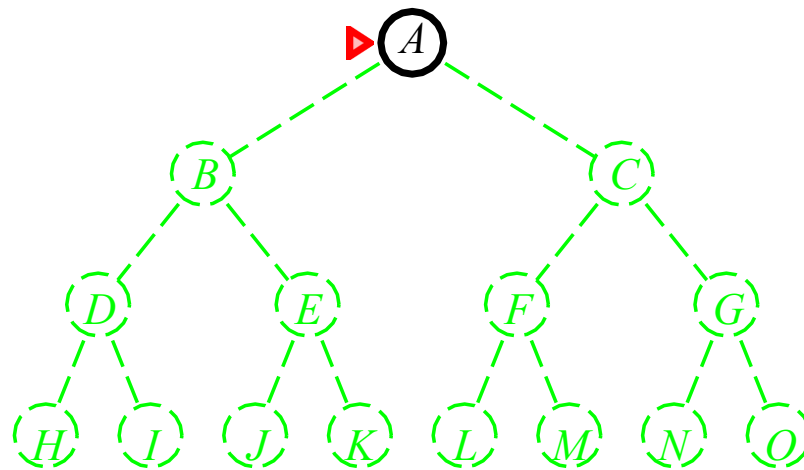


Depth-first search

Expand deepest unexpanded node

Implementation:

frontier = LIFO queue, i.e., put successors at front

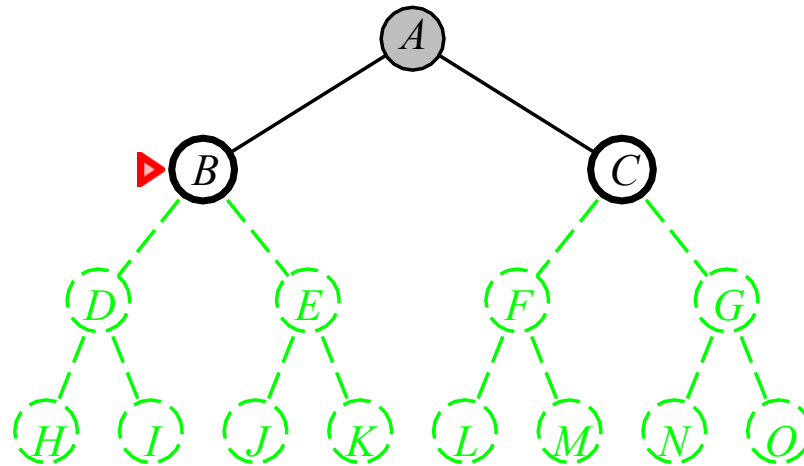


Depth-first search

Expand deepest unexpanded node

Implementation:

frontier = LIFO queue, i.e., put successors at front

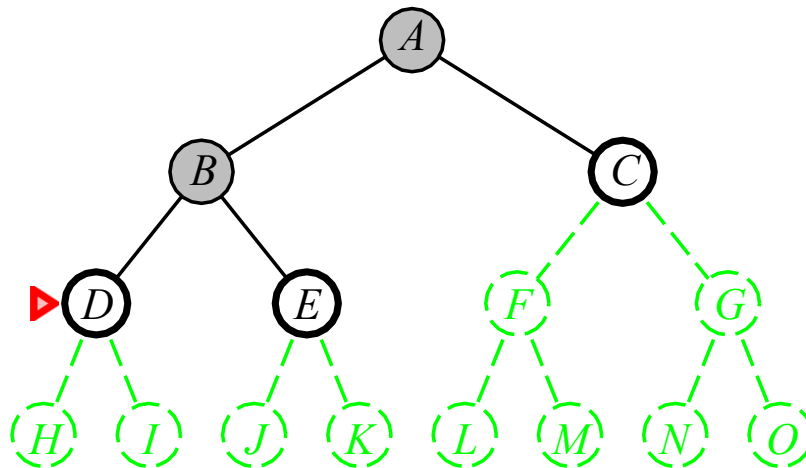


Depth-first search

Expand deepest unexpanded node

Implementation:

frontier = LIFO queue, i.e., put successors at front

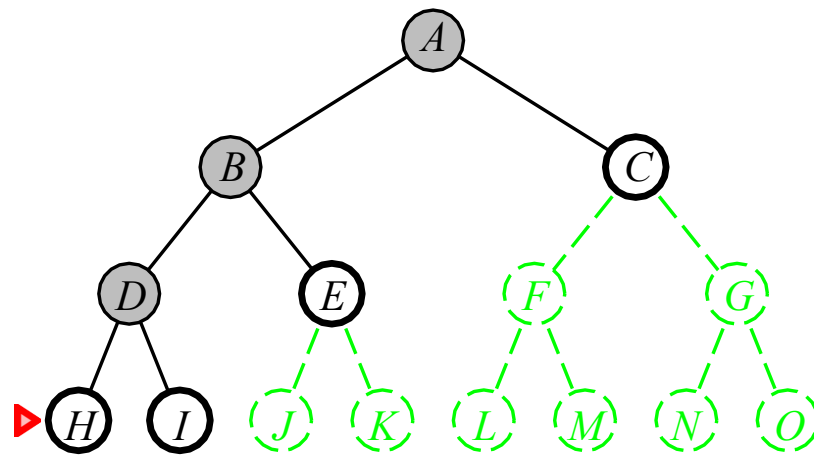


Depth-first search

Expand deepest unexpanded node

Implementation:

frontier = LIFO queue, i.e., put successors at front

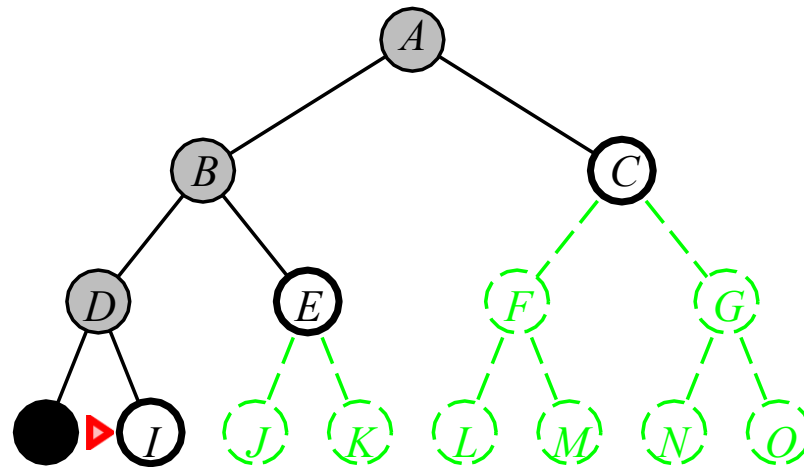


Depth-first search

Expand deepest unexpanded node

Implementation:

frontier = LIFO queue, i.e., put successors at front

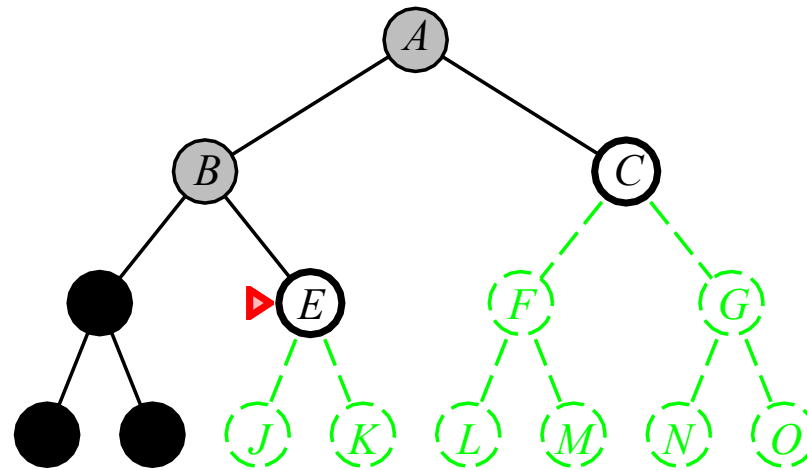


Depth-first search

Expand deepest unexpanded node

Implementation:

frontier = LIFO queue, i.e., put successors at front

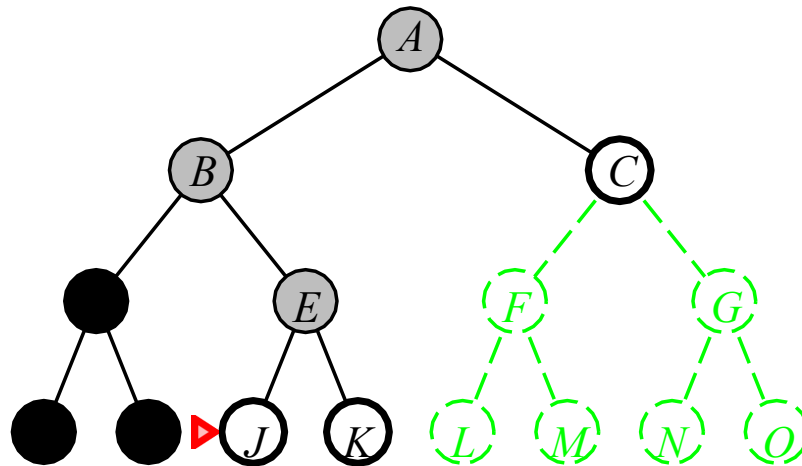


Depth-first search

Expand deepest unexpanded node

Implementation:

frontier = LIFO queue, i.e., put successors at front

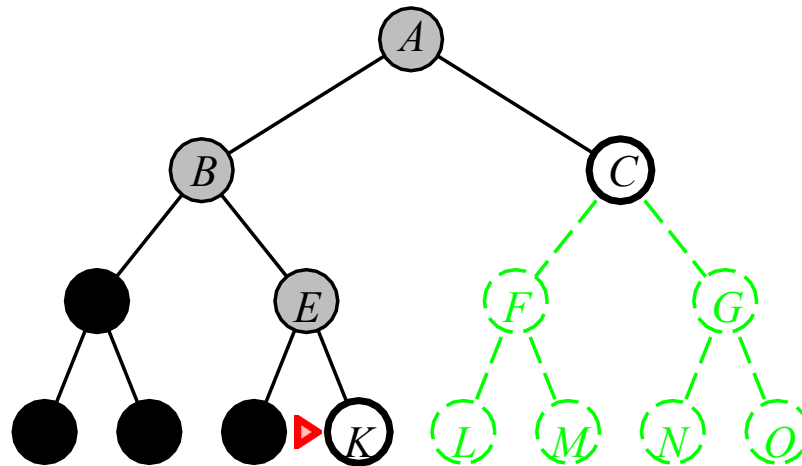


Depth-first search

Expand deepest unexpanded node

Implementation:

frontier = LIFO queue, i.e., put successors at front

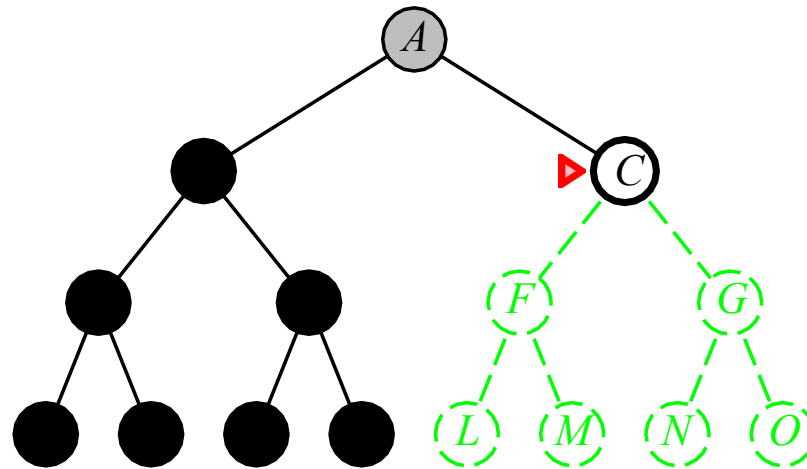


Depth-first search

Expand deepest unexpanded node

Implementation:

frontier = LIFO queue, i.e., put successors at front

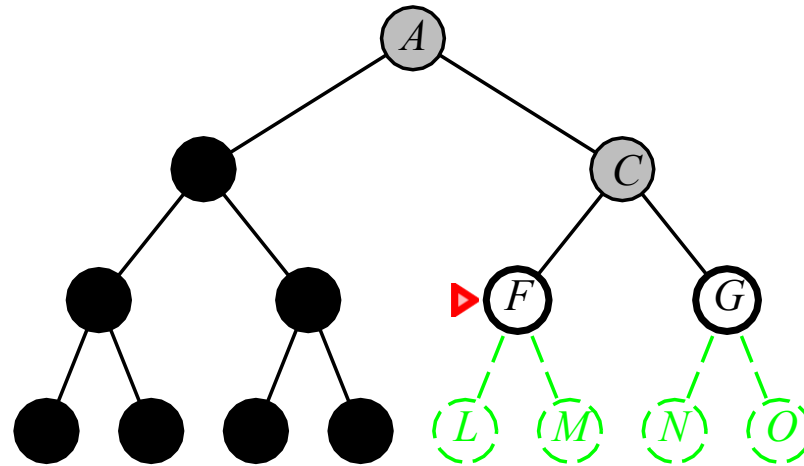


Depth-first search

Expand deepest unexpanded node

Implementation:

frontier = LIFO queue, i.e., put successors at front

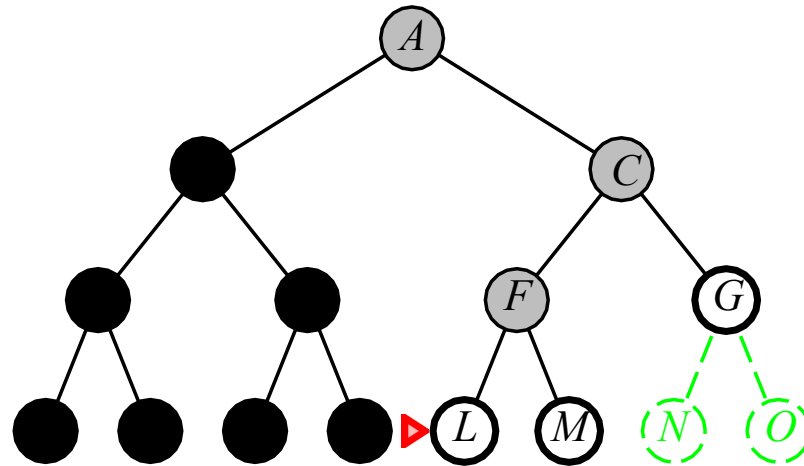


Depth-first search

Expand deepest unexpanded node

Implementation:

frontier = LIFO queue, i.e., put successors at front

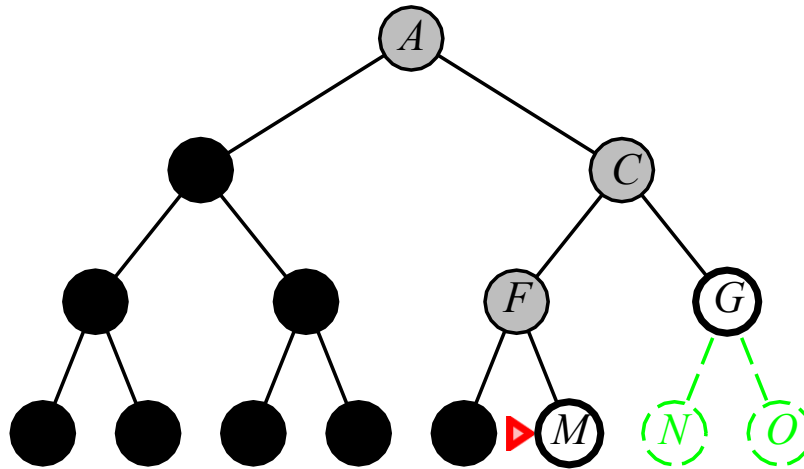


Depth-first search

Expand deepest unexpanded node

Implementation:

frontier = LIFO queue, i.e., put successors at front



Complete? Only in finite state spaces and without loops

Time? $O(b^m)$

Space? $O(bm)$ (linear!)

Optimal? No

A few variants

DFS can be implemented with a fixed depth (**depth-limited search**)

We can try increasing values of depth (**iterative deepening search**)

We can start the search from the initial state forward **and** from the goal state backward (**bidirectional search**).

Issues when navigating the state space

Loops ($A \rightarrow B \rightarrow C \rightarrow A$)

- No loop checks make DFS not complete (unless state space is a tree)

Redundant paths ($A \rightarrow B$ vs. $A \rightarrow C \rightarrow B$)

- You want to expand only the lowest-cost path among the redundant ones

Checking for loops

Store parent for each node

Follow the parent's chain to check if you detect a loop (state already encountered)

No extra memory, only extra time

- **Constant** extra time if you follow the chain for a fixed number of K steps (only detect K -step loops)
- A factor of $O(m)$ if you follow the chain up to the root

Checking for redundant paths

Remember all reached states

- If you *can* store the *reached* table in memory, then why not?

For each state, keep only the best path leading to it

Graph search is a search algorithm that checks for redundant paths

Tree-like search is a search algorithm that does *not* check for redundant paths

Informed (Heuristic) Search Algorithms

Best-first: Expand most desirable unexpanded node

«Desirable» = heuristic/evaluation function $h(n)$

- Provides an estimate of the cheapest cost from **state at node n** to goal

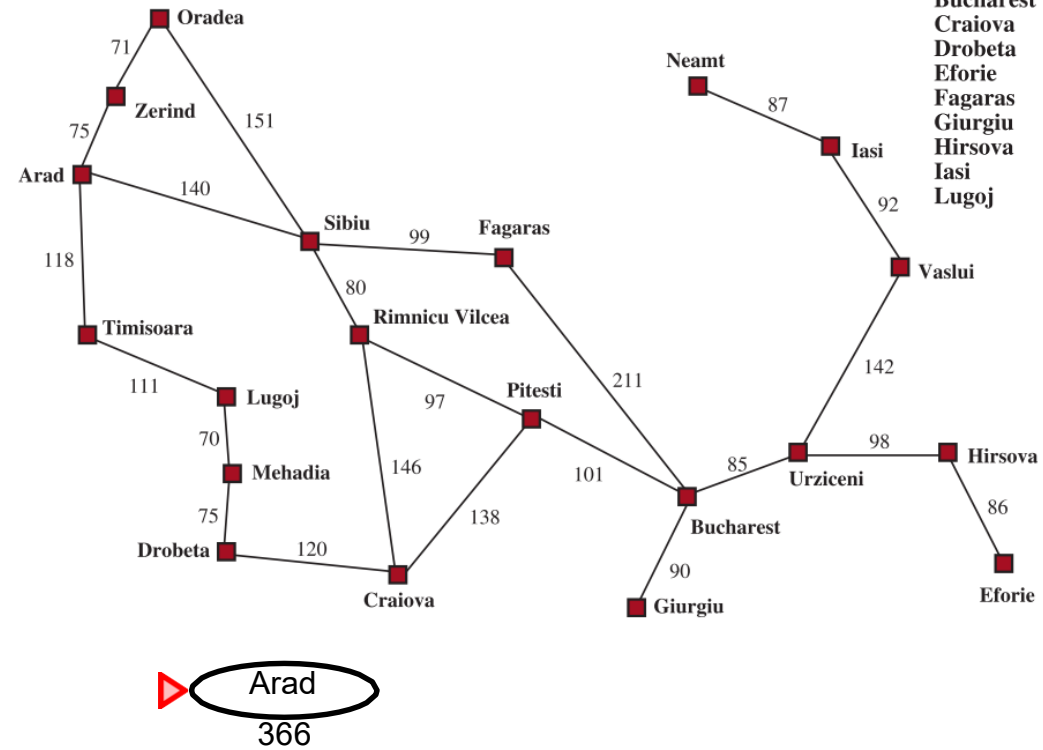
The frontier is a weighted queue sorted by increasing heuristic (cost) value

- Least-cost first

Best-first search algorithms:

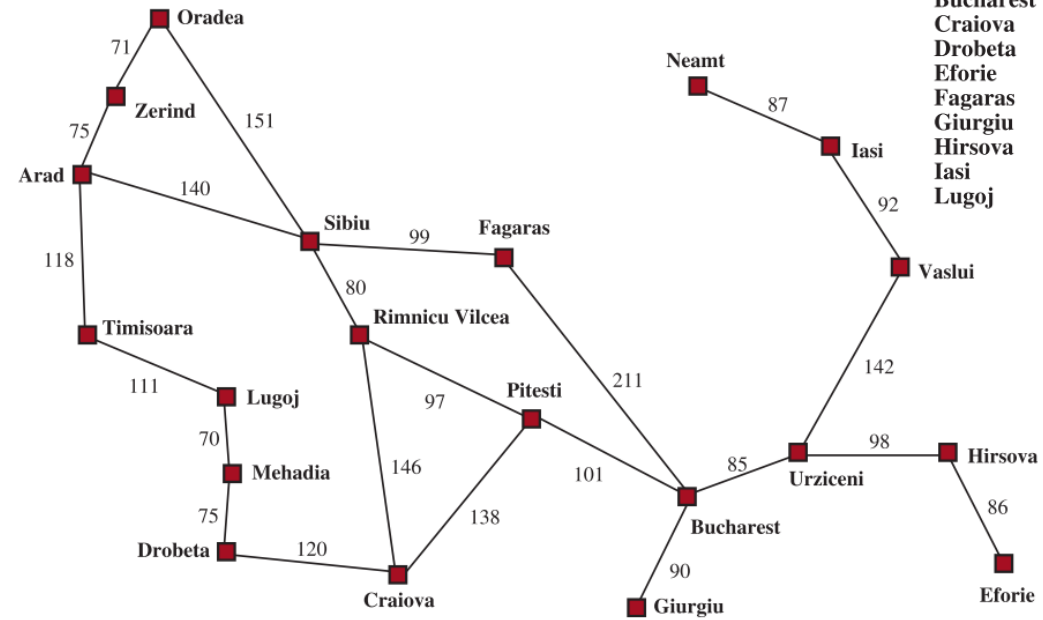
- Greedy search
- A* search

Greedy search

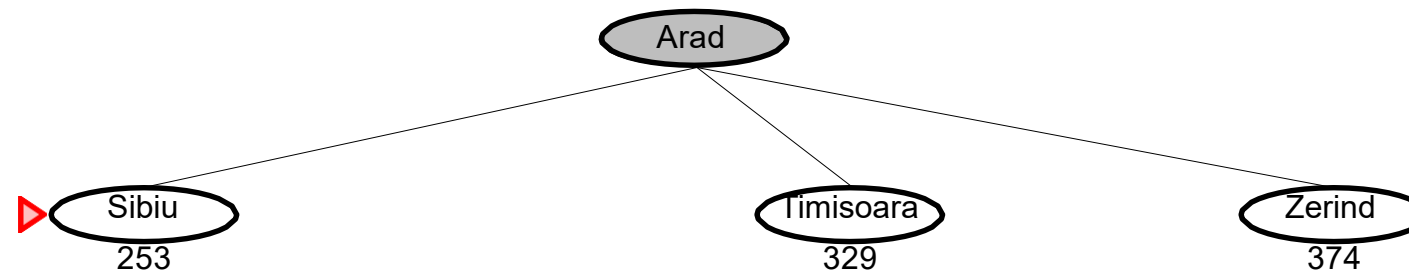


Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

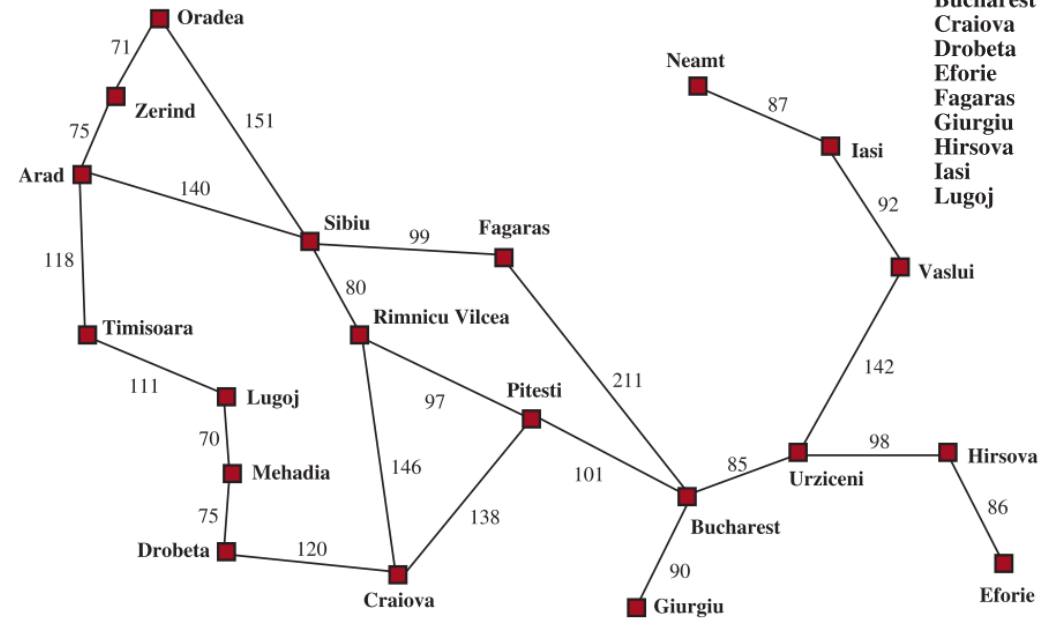
Greedy search



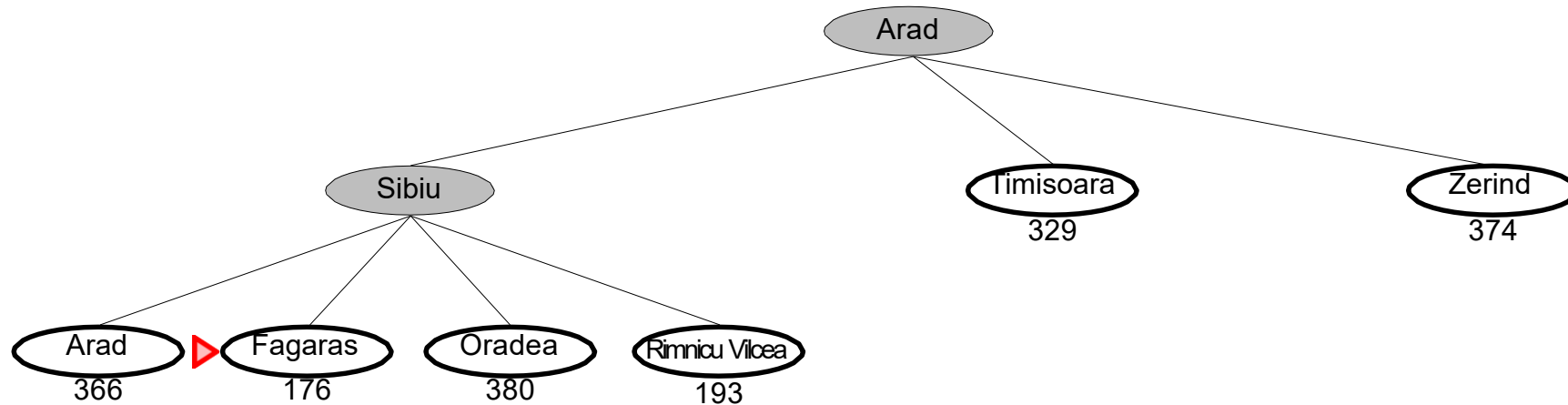
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



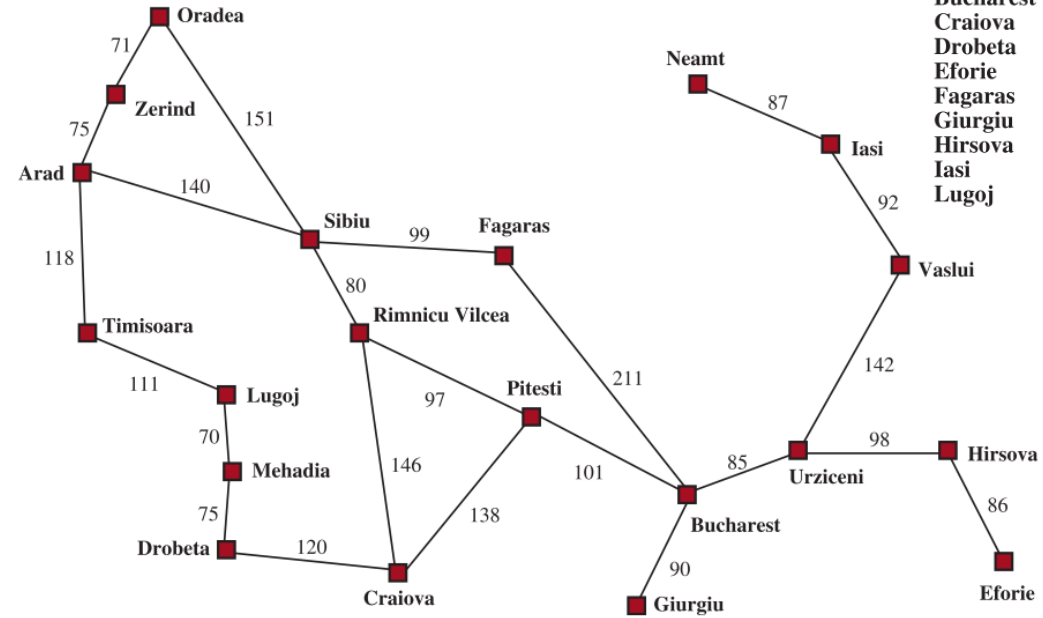
Greedy search



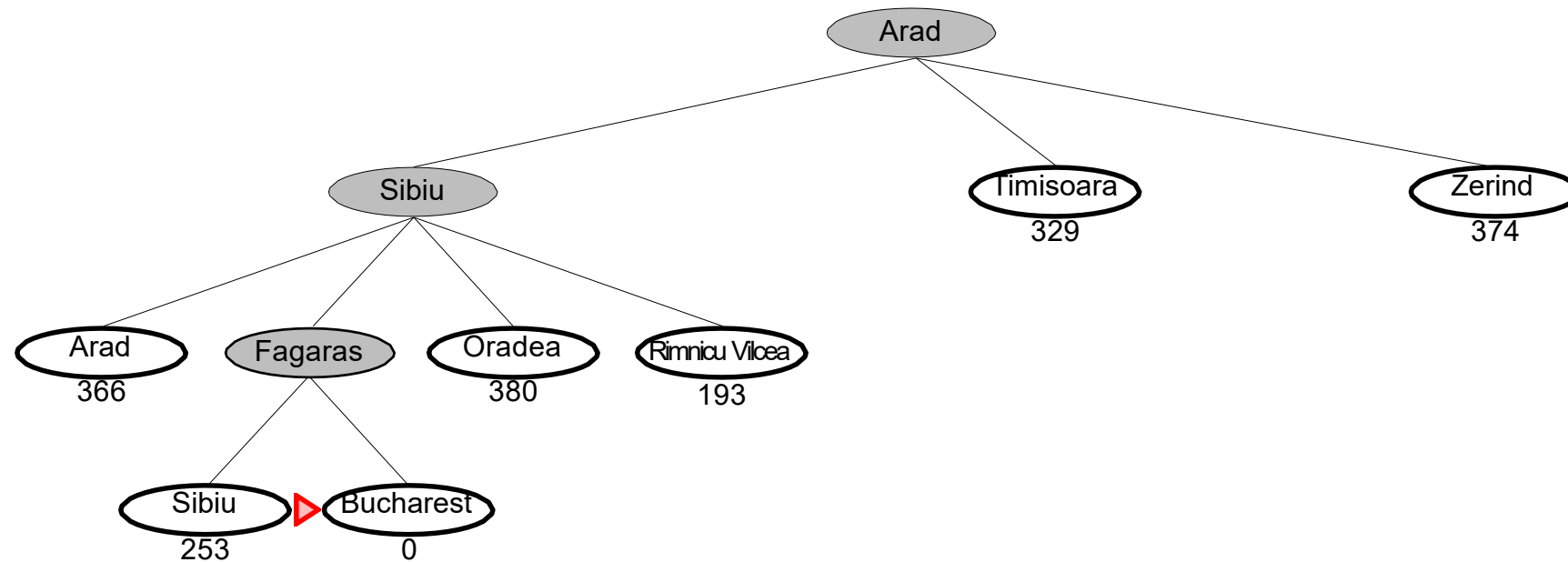
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



Greedy search



Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



Greedy search

Complete? Like DFS

Time? $O(b^m)$

Space? $O(b^m)$

Bad heuristics can drive the search towards deeper region of the tree even though the solution is at a higher level

Optimal? No

(it could skip a step with high cost which then leads to the best solution)

A* search

Idea: avoid expanding paths that are **already expensive**

Evaluation function $f(n) = g(n) + h(n)$

$g(n)$ = cost so far to reach n

$h(n)$ = estimated cost to goal from n (*the “greedy” heuristic*)

$f(n)$ = estimated cost of the best path that passes through n to reach the goal

Space complexity scales with the number of expanded nodes:

A* expands **all** nodes with $f(n) < C^*$

A* expands some nodes with $f(n) = C^*$

A* expands no nodes with $f(n) > C^*$

Time complexity is exponential in the error of h

A* heuristic

A* search uses an **admissible** (optimistic) heuristic

$h(n) \leq h^*(n)$ where $h^*(n)$ is the **true** cost from n .

$h(n) \geq 0$, such that $h(G) = 0$ for any goal G .

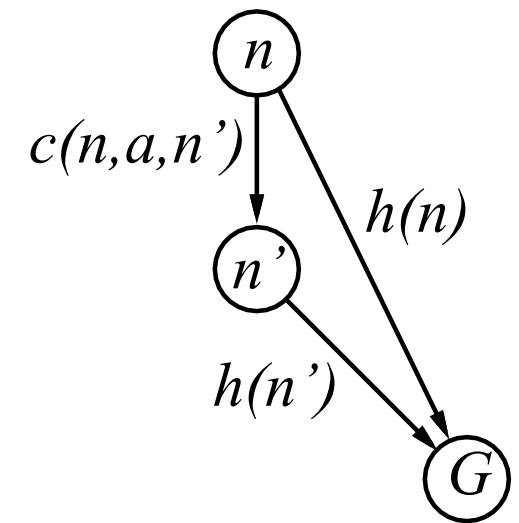
An admissible heuristic never overestimates the distance to the goal.

A* is **complete**

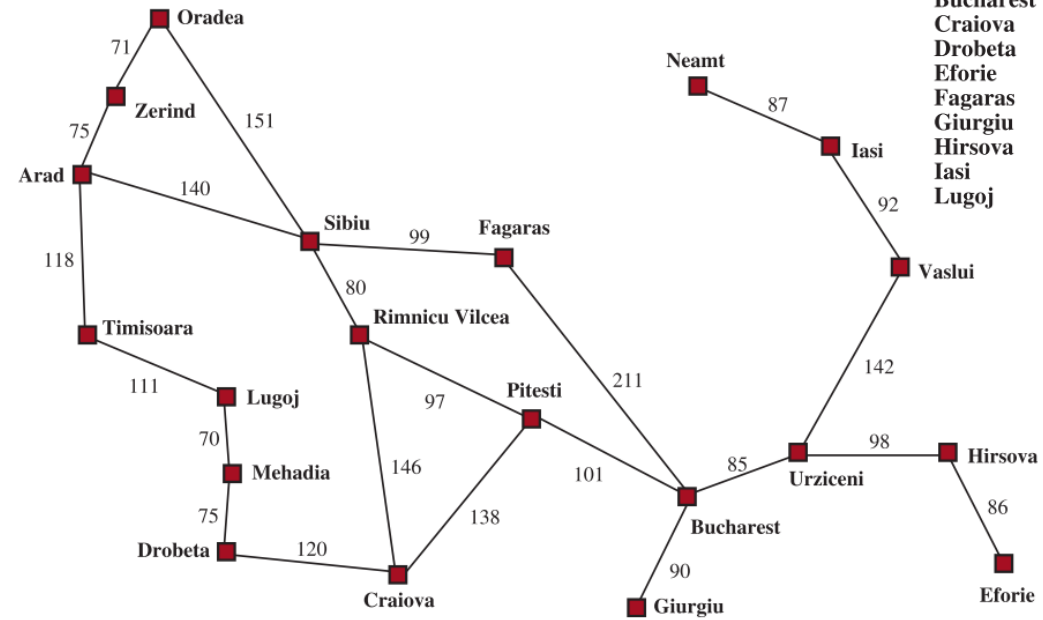
A* is **optimal** if h is admissible

Consistent heuristic (triangle inequality): $h(n) \leq c(n, a, n') + h(n'), \forall n'$

- Stronger than admissibility $\rightarrow$ path costs monotonically increasing
- **once you reach a state, it is on its optimal path already** (visits less nodes)



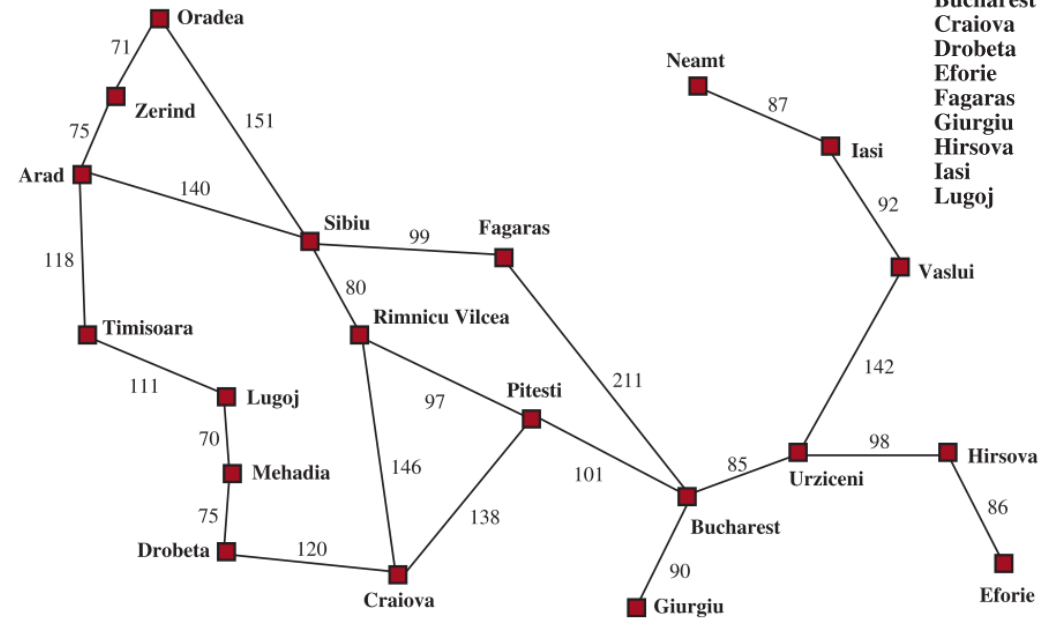
A* simul



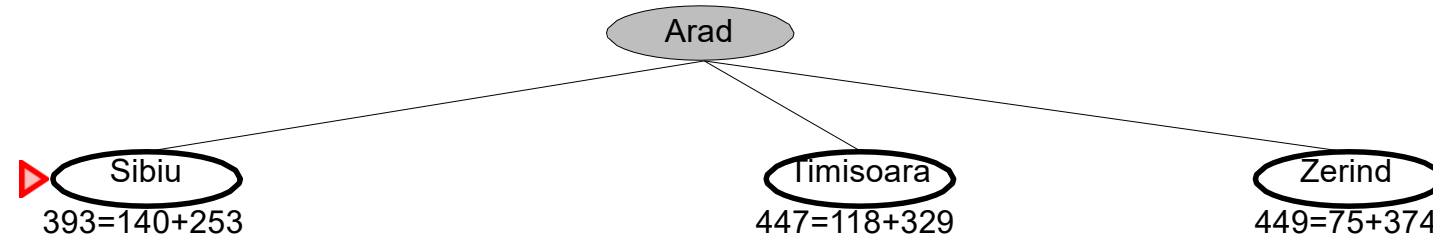
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

▶ Arad
366=0+366

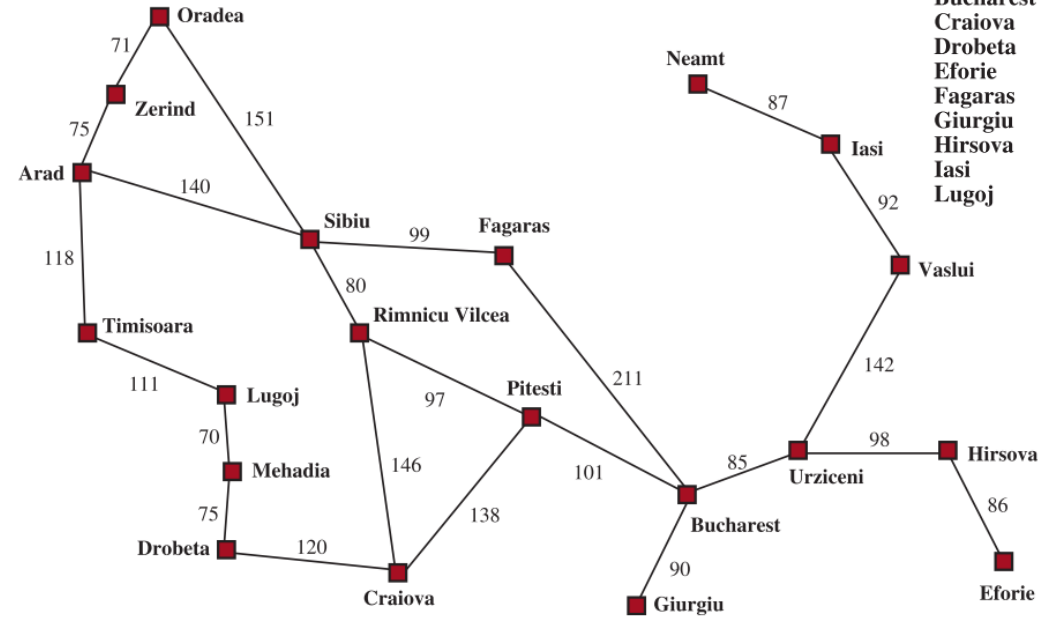
A* simul



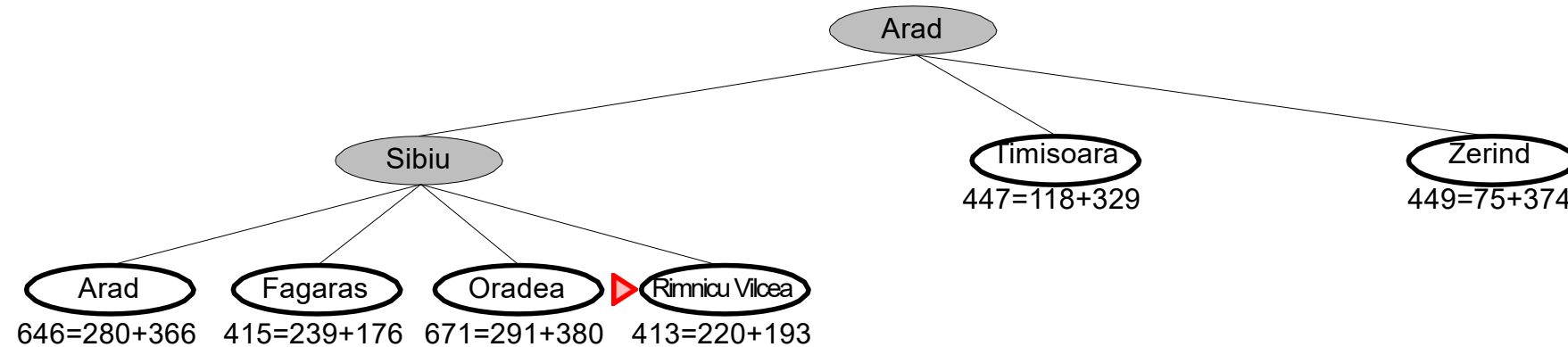
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



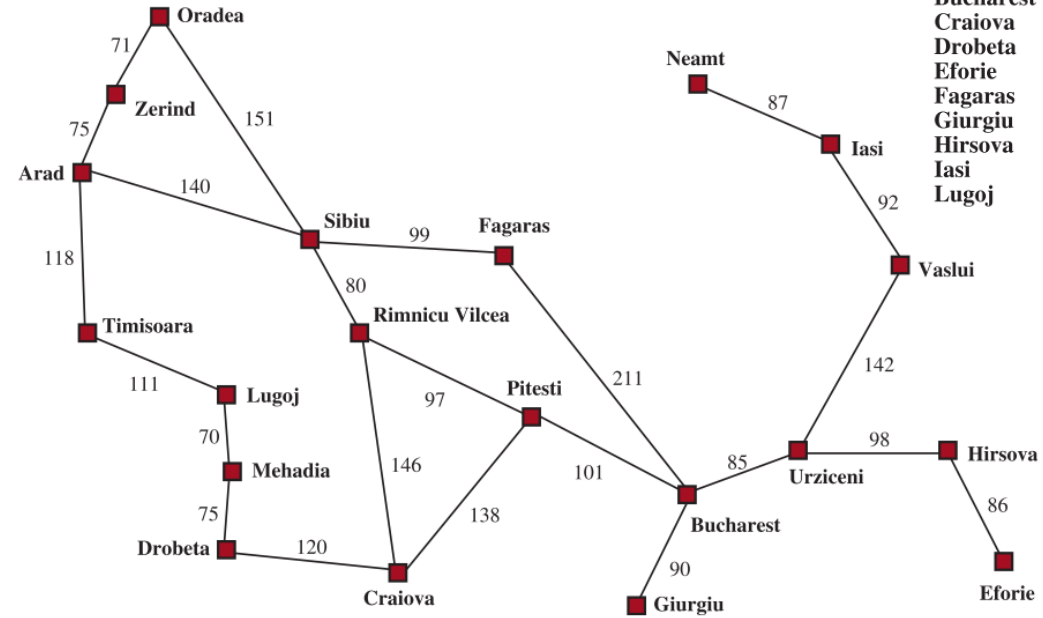
A* simul



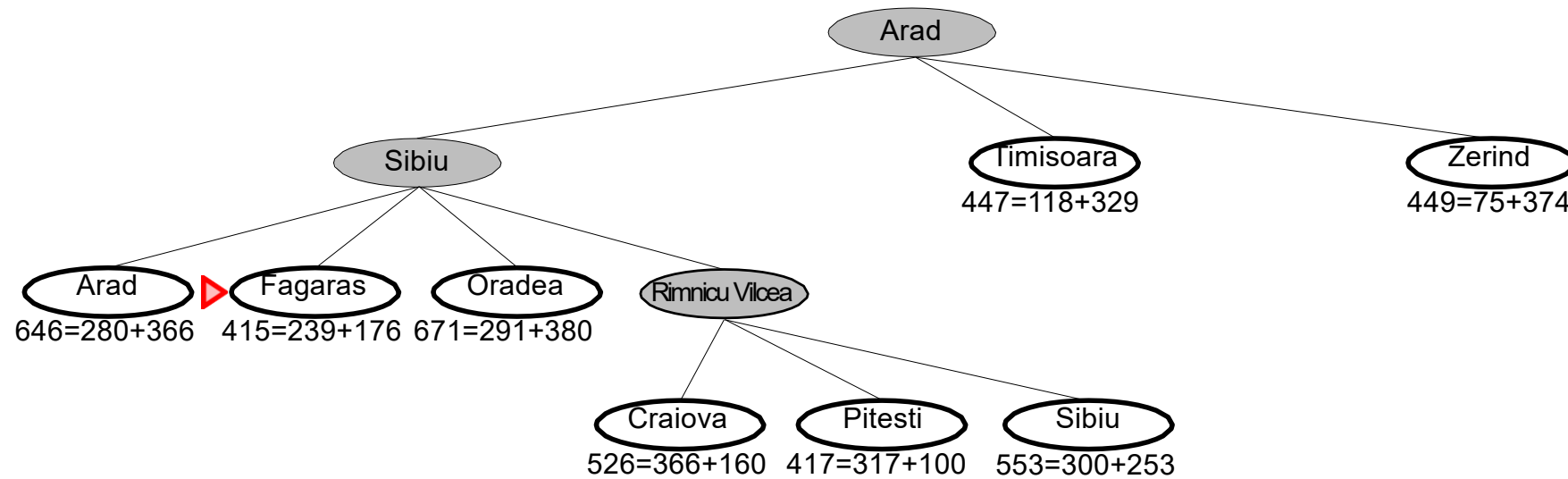
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



A* simul



Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



One proof: optimality of A*

By contradiction: A* returns the path (Start $\rightarrow$ G_2) with cost $C > C^*$

It exists an unexpanded node n on the optimal path

- If all nodes on the optimal path are expanded, A* returns the optimal path

$$f(n) > C^* = g^*(n) + h^*(n)$$

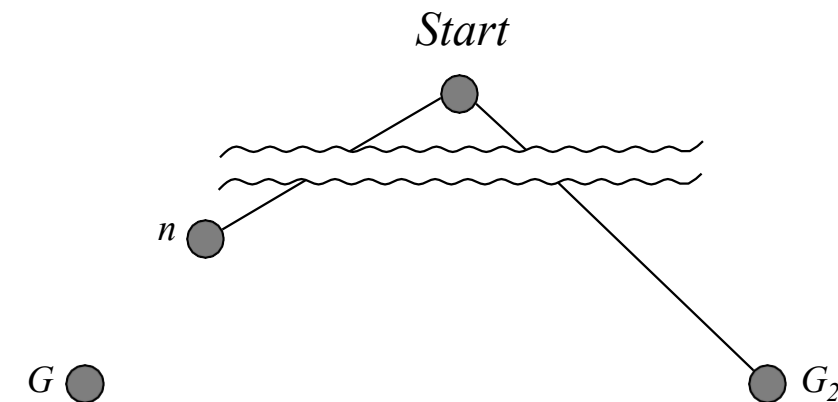
- The first step follows since we found G_2 before n

$$f(n) = g^*(n) + h(n) \leq g^*(n) + h^*(n)$$

- The first step follows since n is on the optimal path
- The last step follows from h being admissible

These two relations are in contradiction $\rightarrow$ there is no such n

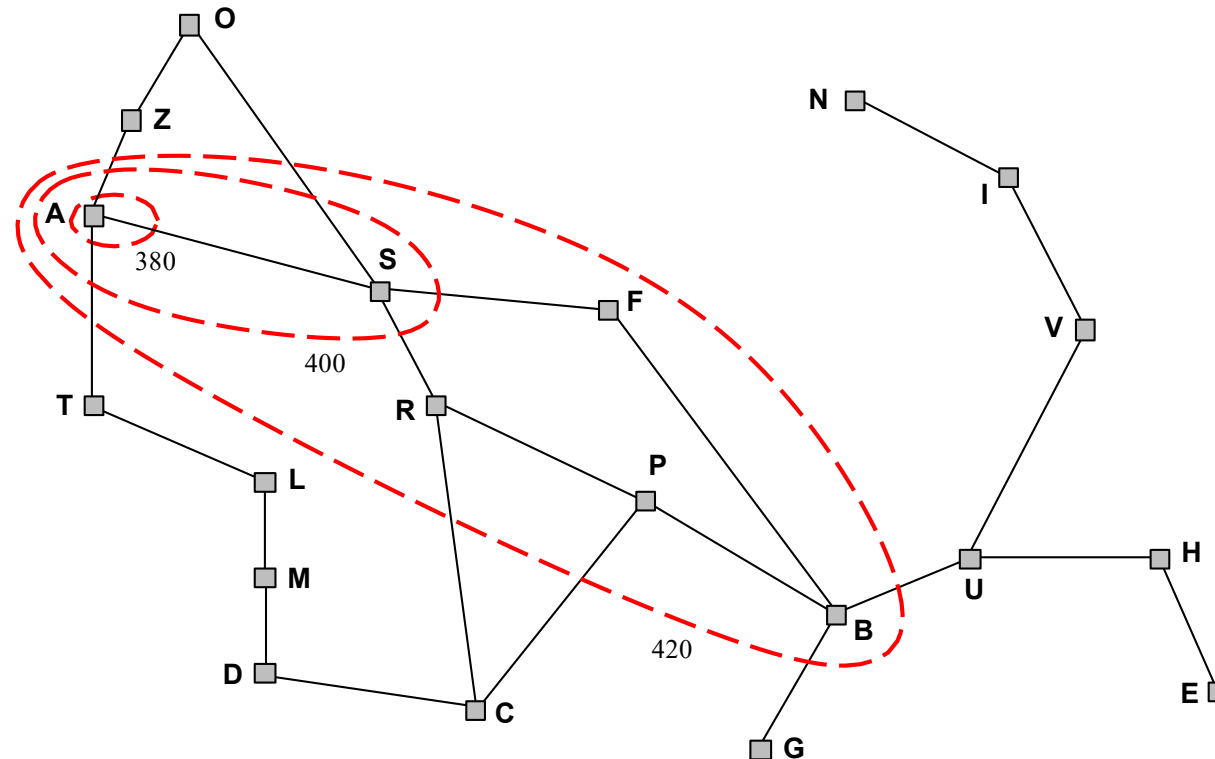
$\rightarrow$ A* returns the optimal path



Search contours

BFS adds «layers» of search».

A* can project much further by adding f-contours with increasing values of f



A* visits too many nodes! Weighted A*

Being complete requires several node expansions.

Satisficing solution: suboptimal, but «good enough».

Weighted A*: $f(n) = g(n) + W h(n)$, $W > 1$

Find solutions with cost between C^* and $W C^*$

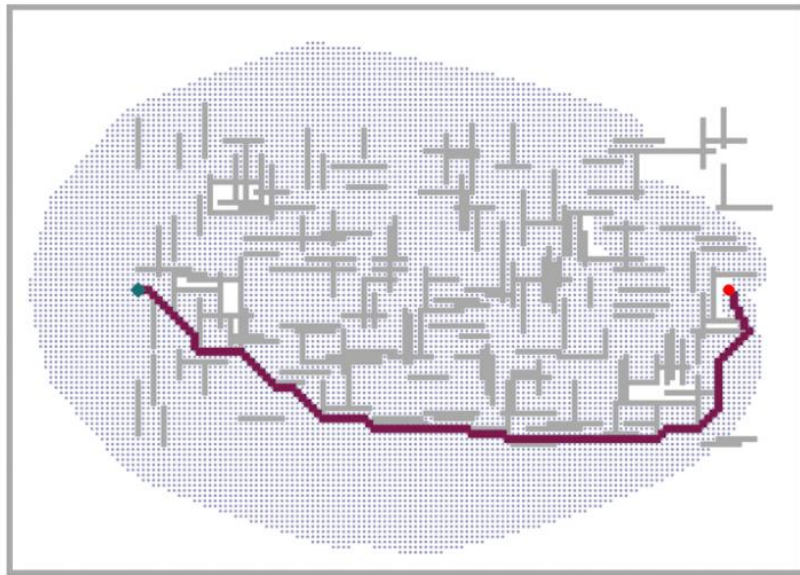
- often close to C^* in practice

Many other variants exist!

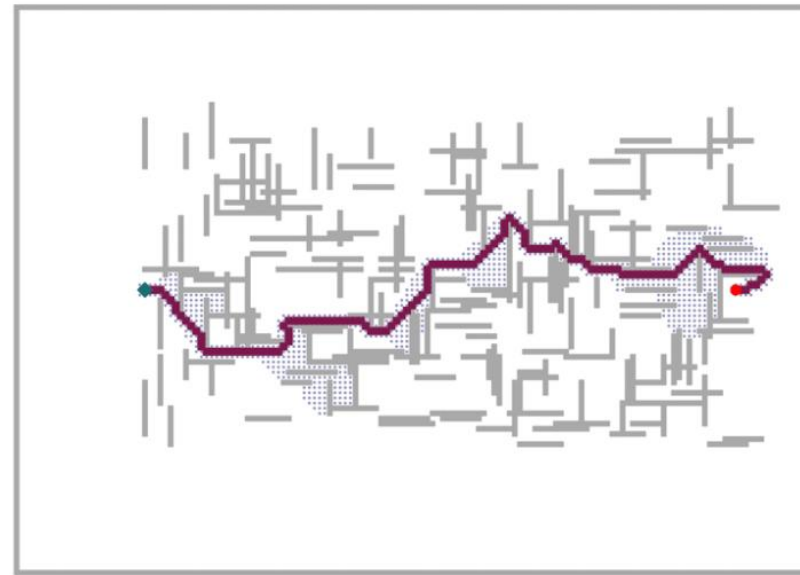
Weighted A*

Weighted A* path (right): 5% increase in cost, 7x save in time

Is it worth to run A*?



(a)



(b)

Play with heuristics: 8-puzzle

$h_1(n) = \text{\#misplaced tiles}$

$h_2(n) = \sum_i \text{Manhattan}(i, \text{target}_i)$

$h_1(S) = 6$

$h_2(S) = 4 + 0 + 3 + 3 + 1 + 0 + 2 + 1 = 14$

h_2 is a better heuristic for search (expand less nodes)

Let h_1 and h_2 be admissible heuristics, then:

h_2 dominates h_1 if $h_2(n) \geq h_1(n) \quad \forall n$

and $\max(h_1, h_2)$ is also admissible and dominates both h_1 and h_2

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

How to design heuristics?

```
from numpy.polynomial import Polynomial

# Define polynomial: x^3 - 6x^2 + 11x - 6
coeffs = [ -6, 11, -6, 1 ] # From constant term to highest degree
p = Polynomial(coeffs)

# Find roots
roots = p.roots()
print("Roots:", roots)
```

Difficult problem, ad-hoc solutions often required
(cfr., reward engineering in reinforcement learning)

Relaxation: find the exact solution of a relaxed (simplified) version of the problem
The optimal solution of a relaxed problem is not better than solution of the true problem
(expand less nodes)

- h_1 is the optimal heuristic (gives the **exact cost** of the solution) for which kind of relaxation of the 8-puzzle problem?

To compare heuristics, measure the **effective branching factor** b^* (the closer to 1 the better)

- Suppose heuristic h finds a solution at depth $d=5$ by using $N=52$ nodes
- Effective branching factor is the branching factor of a tree with 5 layers that contains 52 nodes
- $N + 1 = 1 + b^* + b^{*2} + \dots + b^{*d} \rightarrow$ solve for b^*

d	Search Cost (nodes generated)			Effective Branching Factor		
	BFS	$A^*(h_1)$	$A^*(h_2)$	BFS	$A^*(h_1)$	$A^*(h_2)$
6	128	24	19	2.01	1.42	1.34
8	368	48	31	1.91	1.40	1.30
10	1033	116	48	1.85	1.43	1.27
12	2672	279	84	1.80	1.45	1.28
14	6783	678	174	1.77	1.47	1.31
16	17270	1683	364	1.74	1.48	1.32
18	41558	4102	751	1.72	1.49	1.34
20	91493	9905	1318	1.69	1.50	1.34
22	175921	22955	2548	1.66	1.50	1.34
24	290082	53039	5733	1.62	1.50	1.36
26	395355	110372	10080	1.58	1.50	1.35
28	463234	202565	22055	1.53	1.49	1.36

Learning?

You can **learn** heuristics, they are just functions

- Provided you have enough data

Machine Learning can help you with that

Learning to search: the most popular tool is reinforcement learning

- Now wins easily at board games (and more, e.g., [Dota2](#))
- Lots of compute still involved
- Data may not always be the bottleneck (e.g., self-play).



Upcoming – Local Search
